

Capítulo 16

Calidad de servicio

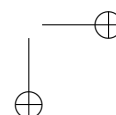
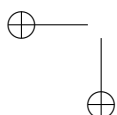
Al terminar este capítulo, entenderás:

Se dice de IP que es un protocolo de «mejor esfuerzo» (*best effort*), que es solo una forma bonita de decir que hace lo que buenamente puede y eso, en función de las condiciones de la red, podría ser nada en absoluto. La integridad, el orden o incluso la propia entrega son prestaciones que IP consigue eventualmente, pero no están en absoluto aseguradas. Aunque TCP sí ofrece estas garantías, no todas las aplicaciones funcionan bien con TCP, e incluso cuando es así, hay otras prestaciones deseables que no ofrece. Por ejemplo, la tecnología TCP/IP básica, tal como la hemos estudiado hasta ahora, no puede ofrecer garantías (ni siquiera lo intenta) una tasa de transferencia mínima o una latencia o *jitter* por debajo de ciertos umbrales.

Una de las limitaciones clave de la tecnología de conmutación de paquetes, en la que se basa IP e Internet, es que no dispone de *control de admisión*, es decir, nunca niega una nueva solicitud de envío o conexión por mucho que la interred ya esté sufriendo una carga excesiva o incluso congestión.

El control de admisión es la característica típica de las redes de conmutación de circuitos, como la telefonía. La red telefónica puede cuantificar la cantidad de recursos disponibles y rechazar el servicio solicitado por un cliente si estima que no lo va a poder satisfacer adecuadamente: el clásico aviso de «todas las líneas están ocupadas». La conveniencia del control de admisión es fácil de entender, pero veremos que hay otras formas no tan obvias de mejorar el servicio.

La Quality of Service (QoS) engloba los mecanismos que tratan de garantizar determinado nivel en los parámetros de tráfico para los servicios que ofrece una interred de conmutación de paquetes. La idea que subyace en la QoS es que la red debería asignar sus recursos en función de las necesidades de cada usuario, tipo de flujo o servicio, bajo la premisa que no todos ne-



cesitan las mismas prestaciones. Por ejemplo, una llamada de voz requiere baja latencia, pero puede aceptar cierta pérdida de mensajes, mientras que la descarga de un archivo puede tolerar alta latencia, pero ninguna pérdida de datos. Fíjate que QoS no implica cotas muy exigentes de rendimiento, solo garantizar ciertos mínimos, que no tienen por qué ser altos, pero que deben cumplirse.

Los parámetros QoS básicos de un flujo se han comentado ya: tasa de transferencia, latencia, *jitter* y tasa de pérdida de paquetes —o confiabilidad en general. Es importante entender que QoS es un juego de suma cero: los recursos son limitados (y normalmente conocidos) y los que consume un flujo no están disponibles para los demás.

Veamos un ejemplo usando la tasa de transferencia, que es un parámetro fácil de entender. Imagina un enlace que, por construcción, está limitado a 100 Mbps: QoS debe garantizar que la suma de las tasas utilizadas por todos los flujos no exceda ese límite. Su papel, por tanto, es doble: asignar a cada flujo los recursos que necesita —aquí, ancho de banda— y evitar que, en conjunto, excedan la capacidad disponible. Sin ese control no existe, cuando se satura el enlace, se descartan paquetes arbitrarios con consecuencias impredecibles; con él, es posible elegir qué paquetes conviene sacrificar para minimizar el daño.

A veces, el límite no es la capacidad física del canal, si no un valor acordado, parte de un contrato comercial entre un proveedor y un cliente, en concreto el ancho de banda contratado se denomina CIR (Committed Information Rate). Imagina un ISP que ofrece a sus clientes conexión de fibra con 1 Gbps de descarga (ese es el CIR). Por encima de eso, el proveedor puede ofrecer un extra si tiene capacidad libre, pero sin garantías. Esa capacidad extra es el **EIR!** (**EIR!**)¹. Este contrato se conoce como SLA (Service Level Agreement) o en español Acuerdo de Nivel de Servicio, y en los casos más exigentes, aparte de la tasa, incluye otros objetivos cuantificables (SLO) y métricas para validar su cumplimiento (SLI).

Empezaremos este capítulo estudiando estos parámetros, su importancia, causas y consecuencias, la forma de medirlos.

Después FIXME

16.1. Tasa de transferencia

La tasa de transferencia mide la velocidad con la que se mueven los datos, ya sea a través de una red, un enlace, un flujo UDP, una conexión TCP, etc.

¹También se suele hablar de PIR, que es $CIR + EIR$

La tasa se mide en bits por segundo (b/s o bps), bytes por segundo (B/s) y sus múltiplos (ver §D.3).

Se distingue entre dos tipos de tasa:

Throughput

o tasa bruta, es la cantidad total de datos transmitidos en un período de tiempo. Además de la carga útil, incluye cabeceras, reconocimientos, retransmisiones o cualquier otro tipo de información de control; todo eso que llamamos sobrecarga de protocolos.

Goodput

o tasa neta, considera solo los datos efectivos, la carga útil procedente del usuario o la aplicación.

En realidad, ambas tasas se pueden medir en distintas capas, considerando diferentes cabeceras y sus mecanismos de control. La tasa bruta de más bajo nivel debería contabilizar hasta las cabeceras de enlace, por lo que tiene que medirse directamente desde la interfaz de red. Veremos más adelante algunas herramientas, como el analizador de tráfico, que puede medirla. `FIXME(tshark, ss)`.

La tasa neta se suele medir sobre el nivel de transporte, es decir, con los datos que se envían o reciben con las primitivas del socket (como veremos en los ejemplos de código) o sobre el nivel de aplicación, que sería el caso en el que únicamente se cuentan los datos relevantes para el usuario, como por ejemplo, un fichero descargado desde un servidor web con HTTP.

También es útil distinguir la tasa en los extremos y en la red:

Tasa de envío (T_s), es la velocidad con la que el proceso emisor entrega datos al SO a través de un socket, que como sabemos, en el caso de TCP no implica necesariamente que esos datos estén saliendo a la red y puede que simplemente se estén almacenando en el buffer de envío.

Tasa de recepción² (T_r), es la velocidad con la que el proceso receptor recoge datos a través de un socket, que con TCP implica leer datos del buffer de recepción.

Tasa de red³ (T_n), es la velocidad a la que los datos se mueven a través de la red. Esta tasa no se puede medir directamente en los extremos, pero se puede estimar a partir de las tasas de envío y recepción.

Lógicamente para cualquiera de las tres, pueden ser útiles tanto la tasa bruta como la neta. Los datos se envían en bloques (segmentos o datagramas) que pueden tener tamaños distintos y que son enviados/recibidos a intervalos posiblemente irregulares. Eso condiciona el cálculo y da lugar a distintos métodos. Veamos los más habituales.

16.1.1. Tasa instantánea

Es el cálculo más sencillo, y probablemente también el menos útil. Consiste en contabilizar únicamente los datos transmitidos desde la medición anterior considerando el tiempo transcurrido desde entonces. Formalmente:

$$R_{\text{inst}} = \frac{\Delta B}{\Delta t} \quad (16.1)$$

donde: R_{inst} es la tasa instantánea (bytes por segundo), ΔB es la cantidad de datos transmitida en el último bloque (bytes), y Δt es el tiempo transcurrido desde el envío del bloque anterior (segundos).

La tasa instantánea puede variar drásticamente de un mensaje al siguiente. Se dice que es muy «sensible al ruido», y por eso raramente resulta útil.

En Python la tasa de envío neta instantánea podría calcularse como en el siguiente fragmento⁴. Fíjate que usamos `time.monotonic()` en lugar de `time.time()` para que el cálculo de los delta no se vea afectado por ajustes en el reloj del sistema.

```
while 1:
    last_time = time.monotonic()
    data = get_data()
    sock.sendall(data)
    byte_rate = len(data) / time.monotonic() - last_time
    print(byte_rate)
```

16.1.2. Promedio acumulado (CA)

En el otro extremo de la tasa instantánea está el promedio total acumulado o CA (Cumulative Average). Consiste simplemente en calcular la media de toda la transmisión hasta el momento, es decir, el total de datos transmitidos partido por el tiempo transcurrido desde el inicio.

$$R_{\text{avg}}(t) = \frac{B(t) - B(t_0)}{t - t_0} \quad (16.2)$$

donde: $R_{\text{avg}}(t)$ es la tasa media acumulada (bytes por segundo), $B(t)$ es la cantidad total de datos transmitida hasta ahora (bytes), $B(t_0)$ es la cantidad total de datos transmitida en el instante 0, y $t - t_0$ es el tiempo transcurrido desde el comienzo (segundos).

Lógicamente sufre del efecto contrario a la tasa instantánea. No se adapta en absoluto a las variaciones (nada reactivo), por lo que una reducción o aumento momentáneo de la tasa queda completamente enmascarado.

⁴Adaptarlo a la tasa de recepción es directo

En Python:

```
start_time = time.monotonic()
sent = 0
while 1:
    data = get_data()
    sock.sendall(data)
    sent += len(data)
    elapsed = time.monotonic() - start_time
    byte_rate = sent / elapsed
    print(byte_rate)
```

LISTADO 16.1: Promedio acumulado

16.1.3. Media móvil simple (SMA)

Una técnica intermedia que ofrece información actualizada, pero más estable que la tasa instantánea, es la ‘media móvil simple’ o SMA (Simple Moving Average). Se basa en definir una ventana de tiempo que abarca solo los w segundos previos considerando únicamente los datos transmitidos en ese lapso.

$$R_{\text{SMA}}(t) = \frac{B(t) - B(t - w)}{w} \quad (16.3)$$

donde: $R_{\text{SMA}}(t)$ es la tasa media móvil simple (bytes por segundo), $B(t)$ es la cantidad total de datos transmitida hasta ahora (bytes), $B(t - W)$ es la cantidad total de datos transmitida hace w segundos, y w es el tamaño de la ventana (segundos).

Aunque es una medida mucho más reactiva que el promedio acumulado, los nuevos datos tardan algún tiempo ($w/2$ segundos) en reflejarse en la tasa.

En Python:

```
class SMA_RateMeter:
    def __init__(self, window_size=5):
        self.window_size = window_size
        self.window = collections.deque()

    def update(self, sent_bytes):
        now = time.monotonic()
        self.window.append((now, sent_bytes))

        # Eliminar datos fuera de la ventana
        while self.window and (now - self.window[0][0] > self.window_size):
            self.window.popleft()

    @property
    def rate(self):
        if not self.window:
```

```

        return 0

    window_bytes = sum(b for t, b in self.window)
    elapsed = self.window[-1][0] - self.window[0][0]
    return window_bytes / elapsed if elapsed > 0 else 0

sma = SMA_RateMeter(window_size=5)
while 1:
    data = get_data()
    sock.sendall(data)
    sma.update(len(data))
    print(sma.rate)

```

16.1.4. Media móvil exponencial (EMA)

SMA también tiene un problema: los datos nuevos de la ventana tienen el mismo peso que los antiguos, por lo que la tasa no refleja bien las variaciones en el tráfico (poco reactivo). Para paliar ese efecto se puede usar una ‘media móvil exponencial’, o EMA (Exponential Moving Average), que aplica un decaimiento exponencial, asignando menos peso a los datos más antiguos.

Se calcula utilizando la siguiente fórmula:

$$R_{\text{EMA}}(t) = \alpha \cdot R_{\text{inst}}(t) + (1 - \alpha) \cdot R_{\text{EMA}}(t - 1) \quad (16.4)$$

donde: $R_{\text{EMA}}(t)$ es la tasa media móvil exponencial, $R_{\text{inst}}(t)$ es la tasa instantánea en el instante t , y α es el factor de suavizado ($0 < \alpha < 1$).

El factor de suavizado α permite ajustar la reactividad del cálculo. Así un valor de 1 la hace equivalente a la tasa instantánea, y valores menores aumentan el peso de los cálculos anteriores. Un valor típico es $\alpha = 0.1$.

En Python, una implementación básica del EMA podría ser:

```

class EMA_RateMeter:
    def __init__(self, alpha=0.1):
        self.alpha = alpha
        self.rate = 0
        self.last = time.monotonic()

    def update(self, sent_bytes):
        elapsed = time.monotonic() - self.last
        self.rate = self.alpha * sent_bytes / elapsed + (1 - self.alpha) * self.rate
        self.last = time.monotonic()

ema = EMA_RateMeter()
while 1:
    data = get_data()
    sock.sendall(data)
    ema.update(len(data))
    print(ema.rate)

```

16.1.5. Consideraciones sobre el cálculo de tasa

Tampoco EMA es perfecto. Al comienzo de una conexión TCP, el emisor puede enviar —invocando `send()`— una gran cantidad de datos en muy poco tiempo porque en realidad esos datos se almacenan en el buffer de envío y recepción, pero no están siendo aún consumidos realmente por el proceso receptor.

Esto genera cálculos de tasa instantánea (en los que se basa EMA) enormes e irreales, porque a fin de cuentas lo que se está midiendo en esos primeros instantes es la tasa entre el proceso y la memoria local y remota, pero no entre los procesos emisor y receptor, que es lo que proporciona una medida representativa. Estos valores tan altos falsean el resultado, e incluso con el decaimiento exponencial, puede llevar mucho tiempo compensar ese efecto. En esta situación no mejora la reactividad, que acaba siendo tan pobre como la del promedio acumulado.

Hay algunas opciones que pueden ayudar a paliar este problema:

- Ignorar la tasa instantánea los primeros segundos de la conexión (período de calentamiento o *warmup*).
- Descartar lapsos (Δt) demasiado pequeños.
- Utilizar un α bajo para Δt pequeños. Se puede calcular como $\alpha = 1 - e^{-\Delta t/\tau}$ siendo τ una constante que determina la reactividad.
- Reducir el tamaño de los buffers TCP de envío y recepción.
- No usar EMA en el emisor.
- Usar SMA en lugar de EMA durante el calentamiento o si no se necesita alta reactividad.

Aplicaremos algunas de estas técnicas a los ejemplos de este capítulo.

16.2. Control de flujo TCP como limitador de tasa

Sabemos que el control de flujo TCP permite al receptor controlar la cantidad de datos que el emisor le envía de acuerdo al espacio del que dispone (la ventana que anuncia). Aunque no se su objetivo principal, este mecanismo se puede utilizar para conseguir un control de la tasa de transferencia rudimentario. Veamos cómo.

Considerando solo uno de los sentidos de la comunicación, asumamos por un momento que el emisor siempre tiene datos disponibles y los puede enviar siempre que sea posible. Con esta premisa se pueden dar tres situaciones en función de la tasa de recepción (T_r) respecto a la velocidad de la red (T_n). Lógicamente la tasa de emisión (T_s) estará limitada por la menor de las otras dos.

- Si $T_r < T_n$, el buffer de recepción se llena, la ventana se cierra, el buffer de emisión también se llena y el emisor se bloquea (*stall*) en espera de que se libere espacio cuando la ventana vuelva a abrir. En esta situación, la tasa de emisión está limitada por el proceso receptor.
- Si $T_r > T_n$, el buffer de recepción queda vacío y el receptor quedará bloqueado en espera de nuevos datos. En esta situación, la tasa de emisión está limitada por la red.
- Si $T_r \approx T_n$, el buffer de recepción nunca se llena ni queda vacío. Ni el emisor ni el receptor se bloquean.

Lógicamente, los buffers de emisión y recepción existen precisamente porque el tercer caso, aunque deseable, es poco frecuente. Estas tres velocidades: emisión, red y recepción suelen ser dispares y cambiantes. Los buffers amortiguan esas diferencias y permiten un flujo más estable. Sin embargo, obviando otros controles —como el de congestión— cuando la ventana está abierta el emisor TCP envía datos a la máxima tasa posible en ese momento, y eso frecuentemente provoca bloqueos en ambos extremos, lo que influye en parte en que T_r y T_n varíen constantemente. Además el algoritmo de Nagle trata de evitar segmentos pequeños, lo que alarga esos bloqueos. La conclusión es que cuando existe disparidad en el desempeño de emisor y receptor, el mecanismo de control de flujo (y en menor medida el de congestión) provoca un flujo a ráfagas, es decir, momentos en los que se envían datos a la máxima velocidad posible y momentos en los que no se envía nada (porque la ventana está cerrada).

La consecuencia directa de todo esto es que si el receptor limita intencionadamente la tasa de recepción, la tasa media de emisión se verá también limitada, aunque su dispersión (*p. ej.* la desviación típica) será muy alta debido al flujo en ráfagas.

En principio una limitación de tasa de este tipo no es incorrecta, pero cuando ocurre de forma generalizada puede provocar efectos perjudiciales como el aumento de la latencia o el *jitter*. Lo veremos más adelante.

16.2.1. Limitación de tasa en el receptor

Veamos cómo implementar un control básico de tasa en el receptor. Como acabamos de ver, se puede conseguir bloqueando intencionadamente el proceso con `time.sleep()` el tiempo necesario para aproximar la tasa medida a la tasa deseada. El siguiente código calcula la tasa con el método de promedio acumulado (`ca_rate`), pero puedes cambiarlo por alguna de las otras opciones que acabamos de ver si lo prefieres.


```

1 target_rate_kBps = 200 * 1000 # 200 kB/s
2 start_time = time.monotonic()
3 received = 0
4 while 1:
5     data = sock.recv(4096)
6     if not data:
7         break
8
9     received += len(data)
10    elapsed = time.monotonic() - start_time
11    ca_rate = received / elapsed
12
13    if ca_rate <= target_rate_kBps:
14        continue
15
16    adjust_time = (received / target_rate_kBps) - elapsed
17    if adjust_time > 0:
18        time.sleep(adjust_time)

```

LISTADO 16.2: Limitación de la tasa de recepción
(medida con promedio acumulado)

Hasta la **línea 14** es equivalente al Listado 16.1 salvo que aquí estamos calculando la tasa de recepción. Si este cálculo está por debajo de la tasa objetivo (`target_rate_kBps`) no hace nada, pero si está por encima, realiza un ajuste aumentando el tiempo (el denominador de la fórmula 16.2). El ajuste se obtiene como la diferencia entre el tiempo realmente transcurrido y el tiempo que corresponde a la tasa objetivo (**línea 16**).

Vamos a retomar el ejemplo del directorio `🐍/flow-control` que ya utilizamos en el capítulo 11 para ilustrar cómo lograrlo. El emisor (cliente) envía datos al receptor (servidor) tan rápido como puede, pero el receptor limita la tasa aplicando la idea que acabamos de ver. Ambos programas calculan la tasa de envío y recepción respectivamente mediante promedio acumulado (§ 16.1.2).

Puedes probar el ejemplo con los siguientes comandos. El segundo parámetro del servidor es la tasa máxima deseada: 200 kB/s.

Cliente

```

~$ ./client.py 127.0.0.1 2000
SNDBUF: 2,626,560 bytes
(-) sent:2,720.0 kB, CA:9176.4 kB/s

```

Servidor

```

~$ ./server.py --limit 200 2000
Client connected: ('127.0.0.1', 42244)
RCVBUF: 131,072 bytes
received:2,724.4 kB, CA:200.1 kB/s

```

FIGURA 16.1: Limitación de la tasa en el receptor
`🐍/flow-control`

Si lo ejecutas, podrás ver que la transferencia en el lado del cliente se detiene durante unos segundos de vez en cuando tal como se ha explicado

en la sección anterior. El receptor no para en ningún momento porque sigue consumiendo los datos que hay en el buffer de recepción. Cuando se libera suficiente espacio, la ventana se abre y el emisor envía nuevos datos durante un tiempo, hasta que el buffer se llena y la ventana se cierra de nuevo.

La Figura 16.2 es un esquema de su funcionamiento y la Figura 16.3 muestra la cantidad de datos enviados por el emisor a partir de la información obtenida de su ejecución (almacenada en el archivo `client-stats.csv`). El diagrama en escalera permite apreciar claramente los períodos de bloqueo (*stall*) debidos al cierre de la ventana.

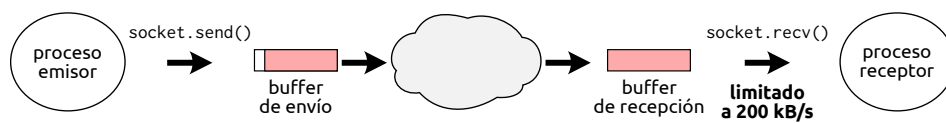


FIGURA 16.2: Limitación de tasa en el receptor

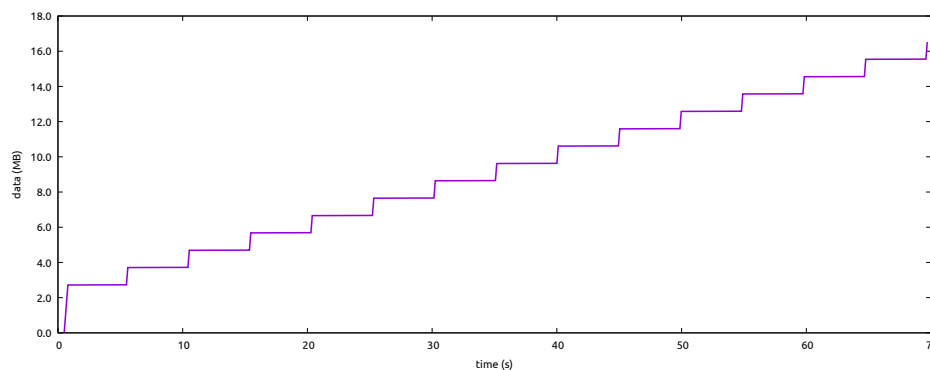


FIGURA 16.3: Datos enviados por el emisor

Aquí puedes apreciar perfectamente el efecto del calentamiento (*warmup*) del que hablábamos en § 16.1.5 respecto a la evolución de la tasa. Debido a que los buffers de envío y recepción son grandes: 2624 kB y 131 kB respectivamente⁵, el emisor al comienzo puede enviar muchos datos ($\sim 2,7$ MB) muy rápido. Una vez llenos los buffers, bloquea por el cierre de la ventana. Con el tiempo la tasa media se irá aproximando a los 200 kB/s impuestos por el receptor. Puedes ver esa evolución en la gráfica 16.4 obtenida también a partir de los datos generados por el emisor.

⁵Esos son los valores por defecto en GNU/Linux para `localhost`.

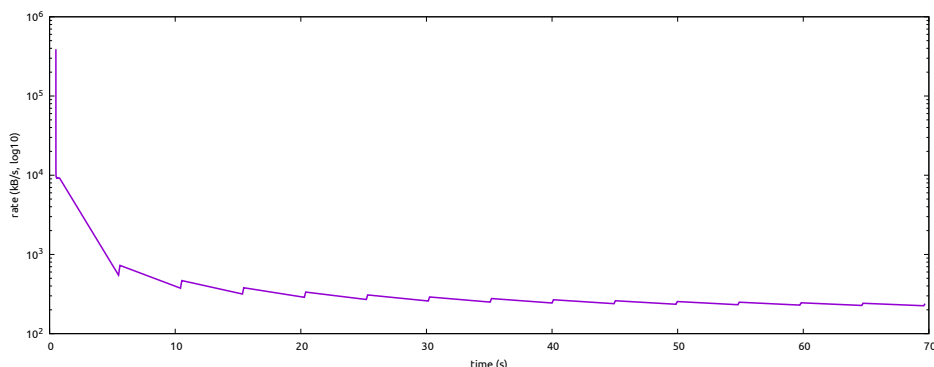


FIGURA 16.4: Tasa de envío medida en el emisor (promedio acumulado)

Los dientes de sierra se deben a que el emisor envía muchos datos en poco tiempo y bloquea (rampa ascendente); cuando reanuda, han pasado unos segundos por lo que la tasa media baja un poco (rampa descendente). Usando otro método para el cálculo de tasa, lógicamente obtendríamos gráficas sin este efecto.

Este mismo servidor proporciona alternativamente un modo diferente de trabajo (que se activa con `-step`). Este argumento permite especificar una cantidad de bytes, que una vez recibidos hacen que el servidor se detenga en espera de que el usuario pulse `ENTER` para continuar. De ese modo puedes comprobar fácilmente cómo el emisor bloquea al llenarse los buffers de envío y recepción y no continúa hasta que quede espacio libre en el receptor. Esta lógica está implementada en el método `step_receiving()` (Listado 16.3).

```

159     def step_receiving(self):
160         input('Press ENTER to receive > ')
161         received = 0
162         while 1:
163             count = 0
164             pending = self.step_size * 1000
165             while count < pending:
166                 data = self.conn.recv(min(4096, pending - count))
167                 if not data:
168                     return
169                 count += len(data)
170             received += count
171
172             input(f'received: {received//1000:,} kB > ')

```

LISTADO 16.3: Servidor con control manual de la recepción

[🔗/flow-control/server.py](#)

En los ejemplos previos el cliente simplemente envía datos sin sentido, y el servidor los descarta. Sin embargo, tienen otro modo que permite indicar

al cliente que consuma datos desde su entrada estándar (con `--stdin`) y al servidor que envíe los datos recibidos a su salida estándar (con `--stdout`). De este modo podemos crear un sistema sencillo para transmitir un fichero mp3 a través de la red, que será consumido en el servidor directamente por un reproductor de audio, vía redirección. Obviamente para que funcione correctamente, el cliente debe enviar un archivo adecuado (un mp3). Este escenario es interesante porque es el reproductor el que determina la tasa de recepción en función de la calidad y compresión del archivo (su *bitrate*). Puedes probar esa posibilidad con estos comandos:

```
~$ ./server.py --stdout --rcvbuf 4000 2000 | mpg123 -q -
~$ ./client.py --stdin --sndbuf 4000 < audio.mp3
```

FIGURA 16.5: Tasa limitada por el reproductor de mp3
 /flow-control-player

Fíjate en las opciones `--sndbuf` y `--rcvbuf` que establecen el tamaño de los respectivos buffer de envío y recepción. Estos valores favorecen períodos de bloqueo más cortos que ayudan a entender cómo funciona la transferencia.

Tanto el cliente como el servidor muestran la tasa con promedio acumulado (CA). El servidor además puede mostrar la tasa SMA si se activa el argumento `--sma` y la tasa EMA con compensación de calentamiento basada en el ajuste de α respecto Δt si se activa el argumento `--ema`. La Figura 16.7 representa ambas tasas para comparación.

```
flow-control$ ./server.py --ema --sma --stdout --rcvbuf 4000 2000 | mpg123 -q -
Client connected: ('127.0.0.1', 52406)
RCVBUF: 8,000 bytes
received:968.7 kB, CA:43.0 kB/s, EMA:40.8 kB/s, SMA:41.0 kB/s
```

FIGURA 16.6: Opciones para medición de tasa
en el servidor: EMA y SMA  /flow-control-player/server.py

En la ejecución de la Figura 16.5 la tasa de recepción ronda los 41 kB/s (328 kbps), muy próxima al bitrate con el que está codificado el mp3 con el que hemos realizado la prueba: 320 kbps.

La consecuencia de tener el reproductor en el receptor es la misma que en los otros ejemplos. Como la tasa de recepción es menor que la tasa de emisión, ambos buffers se llenan y el emisor queda eventualmente bloqueado de manera intermitente. En este caso, también el receptor puede quedar

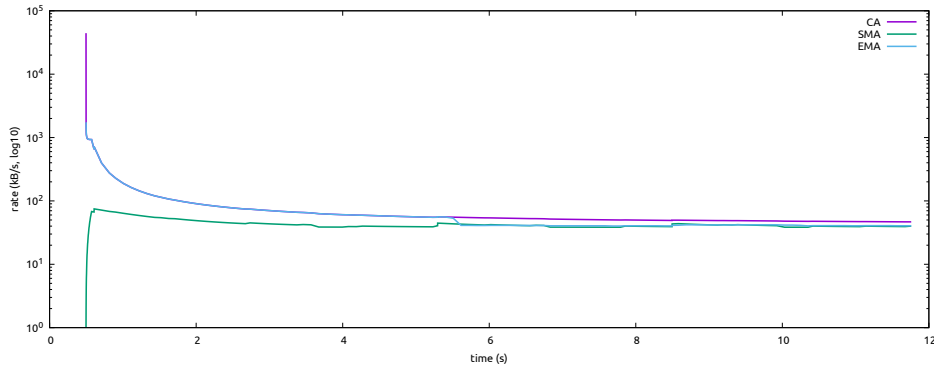


FIGURA 16.7: Comparación de CA, SMA y EMA en el servidor

```
~$ file audio.mp3
audio.mp3: Audio file with ID3 version 2.4.0, contains: MPEG ADTS, layer III, v1,
320 kbps, 44.1 kHz, JntStereo
```

FIGURA 16.8: Información del mp3 utilizado en la prueba

bloqueo porque la entrada estándar del reproductor dispone de un buffer adicional, con lo que la llamada `stdout.flush()` puede bloquear el proceso receptor hasta que haya espacio libre en ese buffer. Ten en cuenta que esto ocurre porque ambos cliente y servidor se están ejecutando en el mismo nodo y se comunican a través de `localhost`. En una conexión remota y en función del ancho de banda disponible, este comportamiento puede variar mucho, hasta el punto de no ocurrir bloqueos si ese ancho de banda es similar al bitrate al que el reproductor consume el flujo.

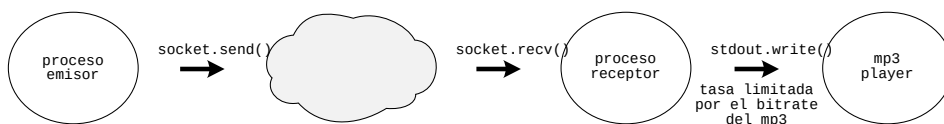


FIGURA 16.9: Datos enviados por el emisor

16.2.2. Limitación de tasa en el emisor

Por supuesto, el emisor puede aplicar exactamente la misma técnica intercalando pausas para conseguir una tasa de emisión concreta. Sin embargo, en este caso la tasa será mucho más estable. El emisor puede enviar bloques de datos más pequeños más a menudo, lo que reduce mucho la dispersión.

El problema es que el emisor no sabe a qué tasa consume el receptor. Si se excede, en algún momento se llenarán los buffers y se bloqueará el emisor. Si es demasiado baja, será el receptor el que se bloquee en espera de datos nuevos y la reproducción del audio se interrumpirá.

16.3. Garantizar prestaciones

Lo primero que debes saber es que eso de «garantizar» es bastante relativo. Lo que puede hacer QoS en una interred IP es priorizar un flujo sobre otro, pero la garantía solo la puede dar si el enlace es capaz de ofrecer las prestaciones totales solicitadas o bien se dispone de algún mecanismo de control de admisión.

Lo más importante es entender que la gestión de rec

Hay dos formas básicas de aplicar prioridades al tráfico:

- **INTSERV** (Integrated Services) identifica flujos y aplica prioridades diferentes a cada flujo.
- **DIFFSERV** (Differentiated Services) utiliza un conjunto de clases de tráfico predefinidas, marca el tráfico con esas clases y después prioriza los recursos en función de ellas.

INTSERV realiza reserva de recursos en cada router antes de comenzar la transmisión. Debido a ello, su escalabilidad es limitada. Los routers que deban manejar miles de flujos deberán mantener información sobre la reserva de recursos de cada uno de ellos. **DIFFSERV** es un enfoque más general y escala mucho mejor porque no requiere configuración previa, el router toma sus decisiones únicamente con la información del paquete.

16.4. Servicios diferenciados

Como cabría esperar, la operación de **DIFFSERV** ocurre principalmente en los routers, aunque los conmutadores y los equipos terminales también pueden participar en determinadas situaciones. El proceso (o *pipeline*) completo se puede resumir en las siguientes tareas:

- **Clasificación**: determina a qué clase de tráfico corresponde el paquete.
- **Marcado**: escribe el identificador de clase en el paquete.
- **Policing**: comprueba si el paquete cumple con los requisitos de QoS.
- **Encolado**: coloca el paquete en la cola que corresponde a su clase.
- **Planificación**: decide de qué cola se extrae el siguiente paquete a reenviar.
- **Shaping**: ajusta la tasa de salida conforme a la contratada.

- Reenvío: transmite el paquete hacia su siguiente salto.

Veamos en qué consiste cada una de ellas.

16.4.1. Clasificación y marcado

El marcado debería realizarse lo más cerca posible del origen del tráfico, aunque no en el propio nodo origen, a menos que se trate de dispositivos terminales cerrados como por ejemplo teléfonos VoIP. Si cualquiera pudiera marcar su tráfico en su propio computador, todos los usuarios elegirían la máxima prioridad y todo el sistema sería inútil.

Lo habitual es que el marcado lo realice el primer switch o router local bajo el control de la organización. Estos dispositivos forman la «frontera de confianza» (*trust boundary*). El proceso utiliza la información del paquete: dirección origen y destino, puerto, protocolo, pero también información de enlace como la etiqueta VLAN, por lo que todos los paquetes de un mismo flujo serán marcados probablemente con la misma clase. Si el paquete ya llegara marcado, el router podría decidir respetar la marca o sobreescribirla si no la considera fiable.

El marcado del tráfico consiste en colocar un identificador de clase —un valor de 6 bits llamado PHB (Per-Hop Behavior)— en el campo DSCP de la cabecera IP⁶.

Los PHB estandarizados son los siguientes:

AC Class Selector (bits xxx000) indica una clase de tráfico codificada en los tres bits MSB (CS1-CS7). DF es en realidad la clase 0 (CS0). Estas clases están definidas en la RFC 2474 [33] y son compatibles con el campo **Precedence** de la especificación original de IP [9]. Las clases, de menor a mayor prioridad, son:

CS1 Tráfico desechable. De hecho, tiene menor prioridad que CS0, *p. ej.* P2P, actualizaciones en segundo plano.

CS0 Tráfico sin garantías *best effort*, la clase por defecto.

CS2 OAM (Operations, Administration and Maintenance), *p. ej.* SSH, SNMP, NTP.

CS3 Video broadcast, *p. ej.* IPTV.

CS4 Vídeo interactivo en tiempo real, *p. ej.* videojuegos, telemetría, telepresencia.

CS5 Señalización de voz y vídeo, *p. ej.* SIP, H.323, H.245.

CS6 Control de la red y enrutamiento, *p. ej.* OSPF, BGP, STP.

CS7 Control crítico / reservado.

⁶El campo *Traffic Class* de IPv6 cumple la misma función

AF_{xy} Assured Forwarding (bits xxx100) indica una de cuatro clases con tres niveles de descarte. Se representan como AF_{xy}, siendo *x* la prioridad de reenvío (1-4) e *y* la tolerancia que ese tráfico tiene al descarte (1-3). Por ejemplo, AF41 indica alta prioridad (4) y baja tolerancia al descarte (1).

EF Expedited Forwarding (bits 101110) es el tráfico de máxima prioridad, RTP/VoIP.

Estos modos diferentes de codificar las clases en realidad se intercalan. La RFC 4594 [34] define un conjunto de 12 perfiles de tráfico, que puedes ver resumidos en el Cuadro 16.1.

Clase de tráfico	PHB / DSCP	Uso típico
Control de red	CS6	Protocolos de encaminamiento
Telefonía	EF	Voz sobre IP
Señalización	CS5	Señalización de llamadas
Videoconferencia	AF41/42/43	Videoconferencia interactiva
Interactivo en tiempo real	CS4	Vídeo interactivo, juegos en red
Streaming multimedia	AF31/32/33	Streaming de vídeo y audio
Vídeo broadcast	CS3	Vídeo broadcast en tiempo real
Datos de baja latencia	AF21/22/23	Transacciones interactivas (ERP, BD)
OAM	CS2	Gestión de red
Datos de alto rendimiento	AF11/12/13	Transferencias masivas, correo, copias de seguridad
Estándar	DF / CS0	Tráfico genérico
Datos de baja prioridad	CS1	Tráfico desechable

CUADRO 16.1: Clases de tráfico recomendadas

16.4.2. Policing

El policing comprueba que el tráfico no excede los valores de tasa establecidos (CIR/PIR) establecidos en el SLA. Aunque evalúa paquetes individuales, tiene en cuenta el comportamiento acumulado del flujo. Si el paquete analizado hace que el flujo exceda la tasa prevista, puede descartarlo o remarcarlo con una prioridad inferior o política de descarte mayor, por ejemplo cambiando un marcado AF41 a AF43.

Para determinar si un paquete es conforme o excede la tasa prevista se suele utilizar el algoritmo de medición *token bucket* (*cubo de fichas*).

El cubo se llena de tokens a una tasa constante de CIR y tiene una capacidad máxima de **CBS**!, que representa el tamaño de ráfaga permitido. Cada paquete que llega consume tokens del cubo (un token por byte). Si el cubo

contiene tokens suficientes, el paquete es conforme y sigue su camino. En caso contrario, el paquete excede y se aplica la acción programada: descartar o remarcar a una prioridad inferior; esta acción no consume tokens. Mientras no hay tráfico, el cubo se sigue llenando hasta alcanzar **CBS!**.

El algoritmo del cubo de tokens deja pasar a los paquetes del flujo si su tasa media es próxima a CIR, pero tolera ráfagas ocasionales siempre que compensen los tokens no consumidos durante la actividad normal. Este es el algoritmo básico y se conoce como *single token bucket* o *single rate two color marker*. Los dos colores representan el resultado: conforme (verde) o excedente (rojo).

La variante **srTCM** (Single Rate Three Color Marker) [35] utiliza dos cubos **C** y **E**. El cubo **C** tiene una capacidad de **CBS!** y juega el mismo papel que en algoritmo básico. El cubo **E** tiene una capacidad de **EBS!** y representa el tamaño de ráfaga excedente. El paquete se marca como conforme (verde) si hay tokens suficientes para su tamaño y es menor que **CBS!**, se marca como excedente (amarillo) si excede **CBS!** pero no **EBS!** y se marca como fuera de perfil (rojo) si excede **EBS!**. Con tres colores, el verde se aplica a un paquete conforme, que se acepta; amarillo a un paquete excedente, que se remarca a una prioridad menor y rojo a un paquete fuera de perfil, que se descarta.

Otra variante más, llamada **trTCM!** (**trTCM!**) [36], utiliza dos tasas: CIR y PIR y dos cubos: **P** y **C** con tasas de llenado PIR y CIR y capacidades **PBS!** y **CBS!** respectivamente. Asigna rojo si **P** no tiene tokens suficientes, amarillo si **P** los tienes, pero **C** no, y verde si ambos tienen tokens suficientes. El amarillo indica que el paquete excede la tasa garantizada, pero no la máxima permitida.

Fíjate que el policer no añade latencia ni jitter al flujo porque nunca encola ni retrasa, aunque como sí descarta, puede provocar retransmisiones.

16.4.3. Encolado

A cada interfaz de salida del router se asocia un conjunto de colas (una por clase) en lugar de simplemente una como vimos en § 13.1. Es una tarea sencilla, simplemente se coloca cada paquete en la cola que le corresponde según la clase con la que está etiquetado.

Los paquetes quedan almacenados en estas colas en espera de que el canal quede libre y el router pueda reenviar el siguiente paquete. Al igual que en un router sin QoS, la ocupación de estas colas crece durante episodios de alta carga. Vimos que en esa situación (ver § 13.7) se utiliza un algoritmo de descarte aleatorio (RED) en lugar de esperar a que la cola se llene.

Sin embargo, con un router QoS, la situación es distinta porque disponemos de información adicional sobre el tráfico. Se aplican algoritmos como WRED (Weighted RED), que considera tanto la clase del paquete como su tolerancia al descarte (la y en las clases AFxy) y que elige paquetes cuyo descarte resulta menos dañino.

Como norma, no se aplica WRED a la cola de máxima prioridad (EF) porque ese tipo de tráfico (*p. ej.* VoIP) tolera muy mal el descarte. Como suele viajar sobre UDP no se retransmite, no soporta ECN ni su descarte ayuda a reducir la congestión.

16.4.4. Planificación

y toma paquetes de esas colas según la prioridad de su clase.

El algoritmo utilizado para determinar de que cola se toma el siguiente paquete es el «planificador» (*scheduler*). Estos son algunos de los planificadores más comunes en orden de complejidad:

FIFO (First In First Out) representa la ausencia de prioridad. Todos los paquetes son tratados del mismo modo, y se reenvían en el orden de llegada. Se podría decir que es el planificador de los routers sin QoS.

SP (Strict Priority) como su nombre indica, aplica una prioridad estricta, es decir, siempre atiende primero a la cola no vacía de mayor prioridad. Muy simple, pero produce inanición para las colas de menor prioridad es episodios de alta carga.

WRR (Weighted Round Robin) reparte el ancho de banda disponible en el enlace entre las colas proporcionalmente a su prioridad.

WFQ (Weighted Fair Queuing) es similar a WRR pero considera el tamaño de los paquetes para determinar la proporción de ancho de banda consumida por cada cola. De ese modo, una cosa de baja prioridad con paquetes grandes no distorsiona la proporción asignada.

CBWFQ (Class Based WFQ) es un WFQ configurable. Permite al administrador definir las clases y el reparto del ancho de banda.

LLQ (Low Latency Queuing) es el planificador más usado. Es una combinación de SP para tráfico de alta prioridad y baja latencia, y CBWFQ para el resto del tráfico. SP se aplica hasta una tasa máxima configurable, para evitar inanición.

16.4.5. Shaping

16.4.6. Reenvío

[Capítulo incompleto]